Industrial Internship Report on

Console-based Expense Tracker

Prepared by

T. Haripriya

Executive Summary
This report provides details of the Industrial Internship provided by upskill Campus and The IoT Academy in collaboration with Industrial Partner UniConverge Technologies Pvt Ltd (UCT). This internship was focused on Console-based Expense tracker statement provided by UCT. We had to finish the project including the report in 4 weeks' time. My project was **Console-Based Expense Tracker**, developed using **Core Java**. During the four-week internship, I designed and implemented the application by adding expense management, category management, filtering, report generation, and file handling features. I also applied Object-Oriented Programming (OOP), Java Collections, and Exception Handling while testing and refining the application to ensure reliable performance. This internship gave me a very good opportunity to get exposure to Industrial problems and design/implement solution for that. It was an overall great experience to have this internship.

TABLE OF CONTENTS

1 Preface

- **Week 1 – Project Planning and Basic Development**
  - Understood the project requirements and identified the key features of the Expense Tracker application.
  - Designed the project structure using Object-Oriented Programming (OOP) concepts.
  - Created the Expense class to represent expense details such as ID, date, amount, category, and description.
  - Developed the basic menu-driven console interface.
  - Implemented the **Add Expense** and **View Expenses** functionalities using ArrayList.
  - Learned the fundamentals of Java classes, objects, constructors, methods, and collections.

- **Week 2 – Expense and Category Management**
  - Implemented **Update** and **Delete Expense** functionalities.
  - Developed the **Category Management** module to add, view, and delete expense categories.
  - Implemented expense search and filtering based on category, date, and amount.
  - Improved the user interface by organizing menus and displaying formatted outputs.
  - Gained practical experience in Java Collections (ArrayList), loops, conditional statements, and method design.

- **Week 3 – Data Persistence and Report Generation**
  - Implemented file handling using BufferedReader and BufferedWriter to store expense and category data permanently.

- Enabled users to save and load expenses from text files.
- Developed **Monthly Expense Reports**, **Category-wise Reports**, and **Expense Summary Reports**.
- Improved data management by integrating file operations with the expense management system.
- Learned Java File Handling, data persistence, and report generation techniques.

- **Week 4 – Testing, Validation, and Project Completion**
  - Added input validation and exception handling to improve application reliability.
  - Performed comprehensive testing of all modules, including expense management, category management, filtering, reporting, and file handling.
  - Fixed minor bugs and improved the overall code structure and user experience.
  - Finalized the Expense Tracker application and prepared the project documentation and internship report.
  - Successfully completed a fully functional console-based Expense Tracker application demonstrating Core Java concepts such as Object-Oriented Programming, Collections, File Handling, Exception Handling, and menu-driven application development.

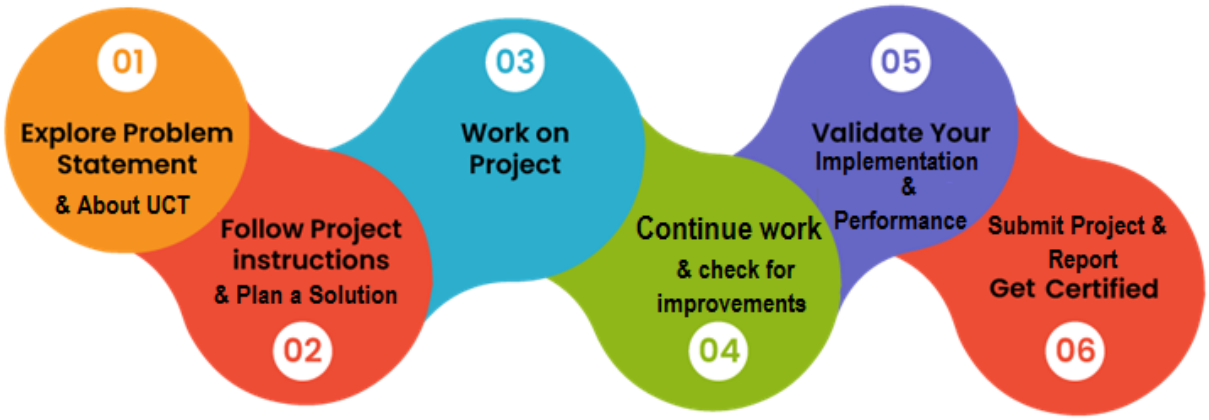
Project Statement

The **Console-Based Expense Tracker** is a Core Java application developed to help users efficiently manage their daily expenses. It enables users to record, update, delete, categorize, and track expenses while generating monthly and category-wise reports. The application uses Object-Oriented Programming (OOP), Java Collections, File Handling, and Exception Handling to provide a simple, reliable, and user-friendly solution for personal expense management with persistent data storage.

Opportunity given by USC/UCT

I sincerely thank USC/UCT for providing me with this valuable internship opportunity. It enhanced my Core Java skills, strengthened my confidence, and offered practical project experience that contributed significantly to my professional and career development.

How Program was planned



Need of internship in career development: A relevant internship plays a vital role in career development by providing practical industry experience, improving technical and problem-solving skills, enhancing confidence, and preparing individuals for future job opportunities through real-world project exposure.

The internship enhanced my Core Java knowledge and practical programming skills. I gained experience in OOP, file handling, exception handling, and problem-solving while developing a real-world project, making it a valuable learning experience.

Thanks to my friend for suggesting me this internship, thus giving me an opportunity to learn in upskill campus. I used various sources of learnings such as books, articles, websites and youtube channels for learning and I am grateful for them.

And for my peers and juniors, stay curious, practice consistently, and embrace every learning opportunity. Real-world projects build confidence, improve skills, and prepare you for a successful career.

2 Introduction

2.1 About UniConverge Technologies Pvt Ltd

A company established in 2013 and working in Digital Transformation domain and providing Industrial solutions with prime focus on sustainability and RoI.

For developing its products and solutions it is leveraging various **Cutting Edge Technologies e.g. Internet of Things (IoT), Cyber Security, Cloud computing (AWS, Azure), Machine Learning, Communication Technologies (4G/5G/LoRaWAN), Java Full Stack, Python, Front end** etc.

i. UCT IoT Platform (uct Insight)

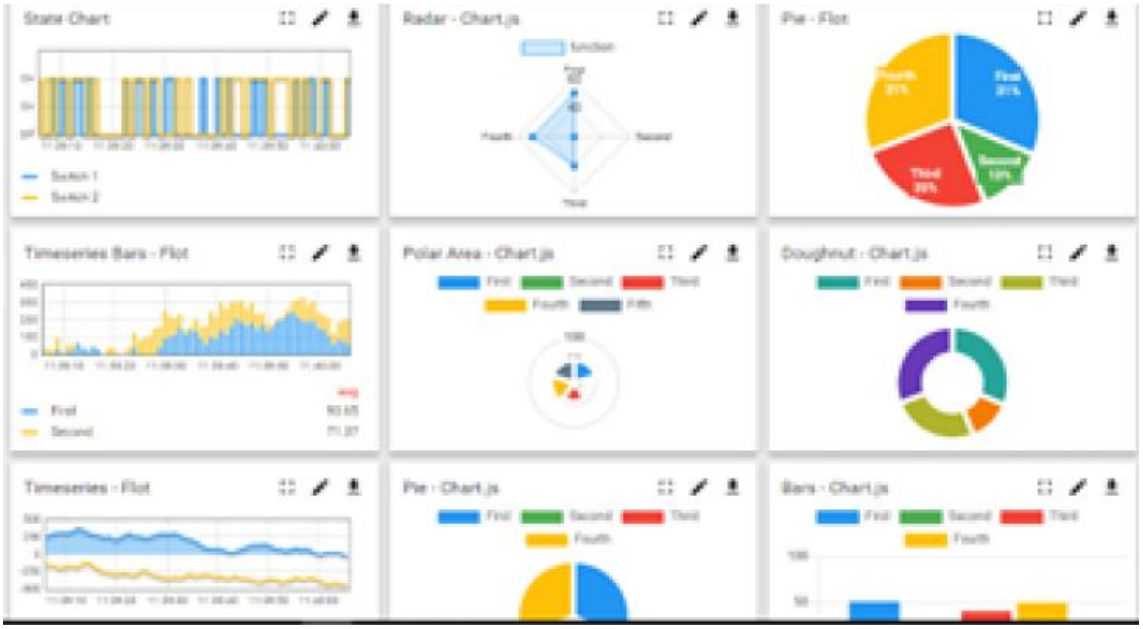
UCT Insight is an IOT platform designed for quick deployment of IOT applications on the same time providing valuable "insight" for your process/business. It has been built in Java for backend and ReactJS for Front end. It has support for MySQL and various NoSql Databases.

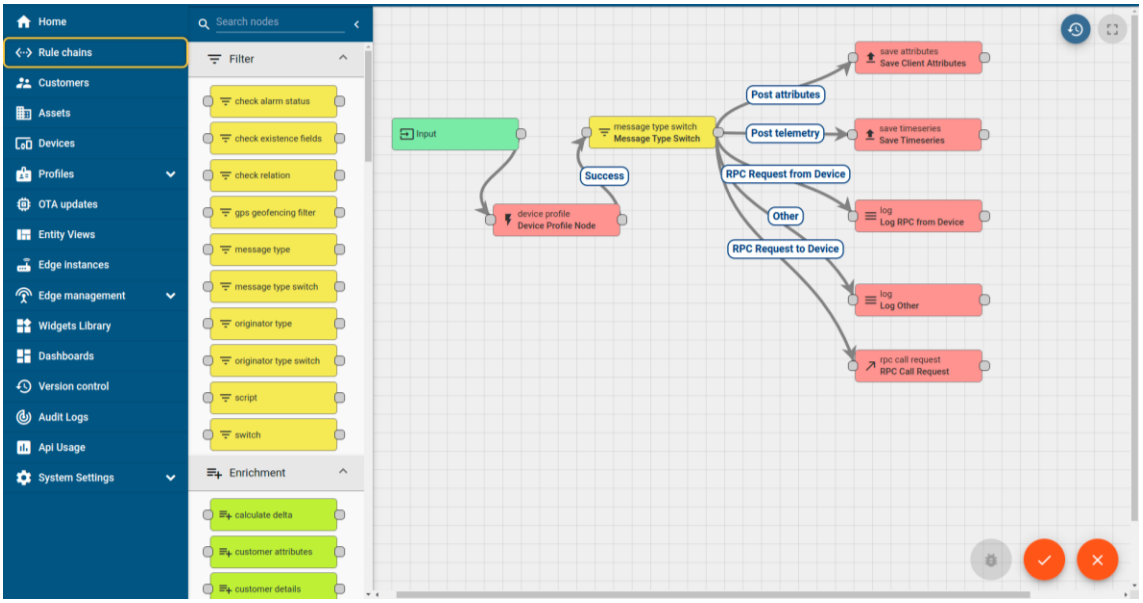
- It enables device connectivity via industry standard IoT protocols - MQTT, CoAP, HTTP, Modbus TCP, OPC UA

- It supports both cloud and on-premises deployments.

It has features to

- Build Your own dashboard
- Analytics and Reporting
- Alert and Notification
- Integration with third party application(Power BI, SAP, ERP)
- Rule Engine





ii. Smart Factory Platform (FACTORY WATCH)

Factory watch is a platform for smart factory needs.

It provides Users/ Factory

- with a scalable solution for their Production and asset monitoring
- OEE and predictive maintenance solution scaling up to digital twin for your assets.
- to unleased the true potential of the data that their machines are generating and helps to identify the KPIs and also improve them.
- A modular architecture that allows users to choose the service that they what to start and then can scale to more complex solutions as per their demands.

Its unique SaaS model helps users to save time, cost and money.

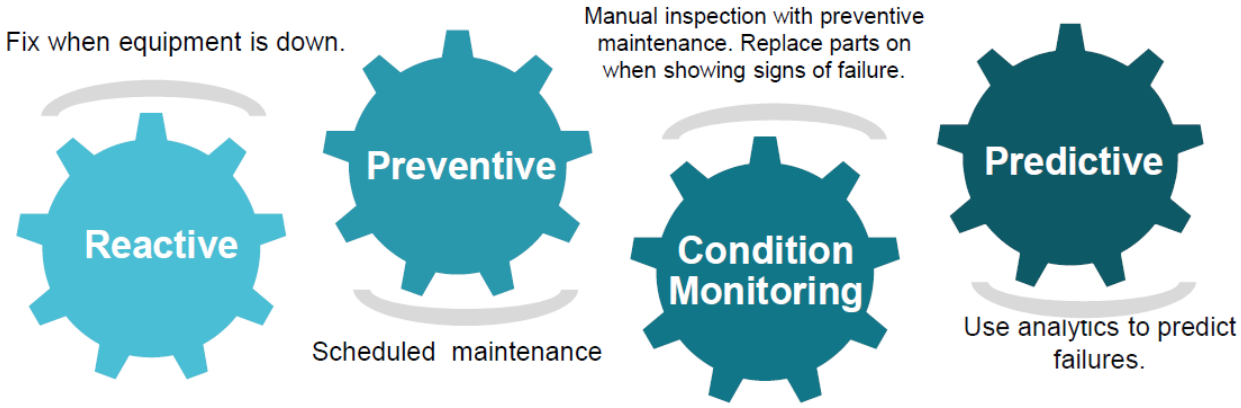
Machine	Operator	Work Order ID	Job ID	Job Performance	Job Progress		Output		Rejection	Time (mins)				Job Status	End Customer
					Start Time	End Time	Planned	Actual		Setup	Pred	Downtime	Idle		
CNC_S7_81	Operator 1	WO0405200001	4168	58%	10:30 AM		55	41	0	80	215	0	45	In Progress	i
CNC_S7_81	Operator 1	WO0405200001	4168	58%	10:30 AM		55	41	0	80	215	0	45	In Progress	i

iii. LoRaWAN based Solution

UCT is one of the early adopters of LoRAWAN teschnology and providing solution in Agritech, Smart cities, Industrial Monitoring, Smart Street Light, Smart Water/ Gas/ Electricity metering solutions etc.

iv. Predictive Maintenance

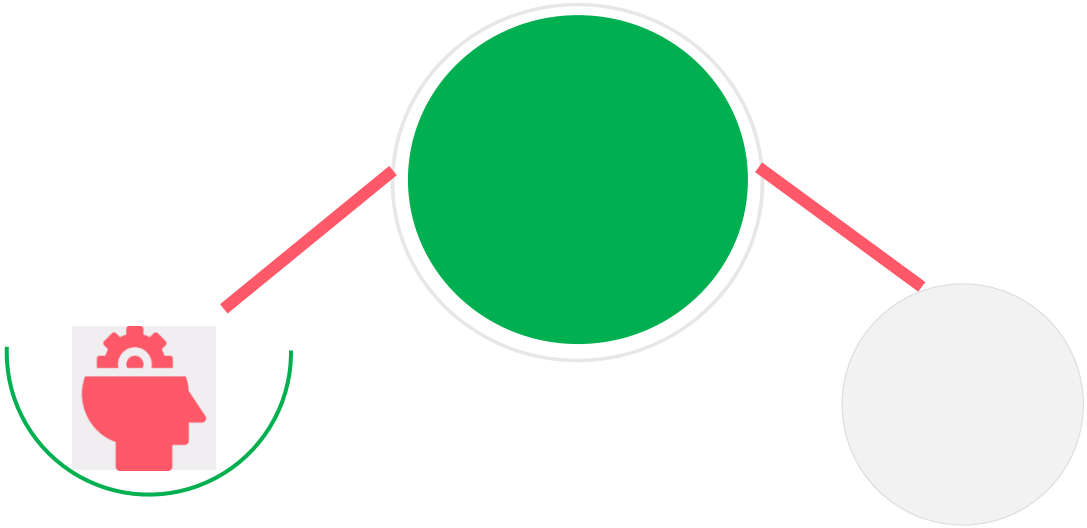
UCT is providing Industrial Machine health monitoring and Predictive maintenance solution leveraging Embedded system, Industrial IoT and Machine Learning Technologies by finding Remaining useful life time of various Machines used in production process.



2.2 About upskill Campus (USC)

upskill Campus along with The IoT Academy and in association with Uniconverge technologies has facilitated the smooth execution of the complete internship process.

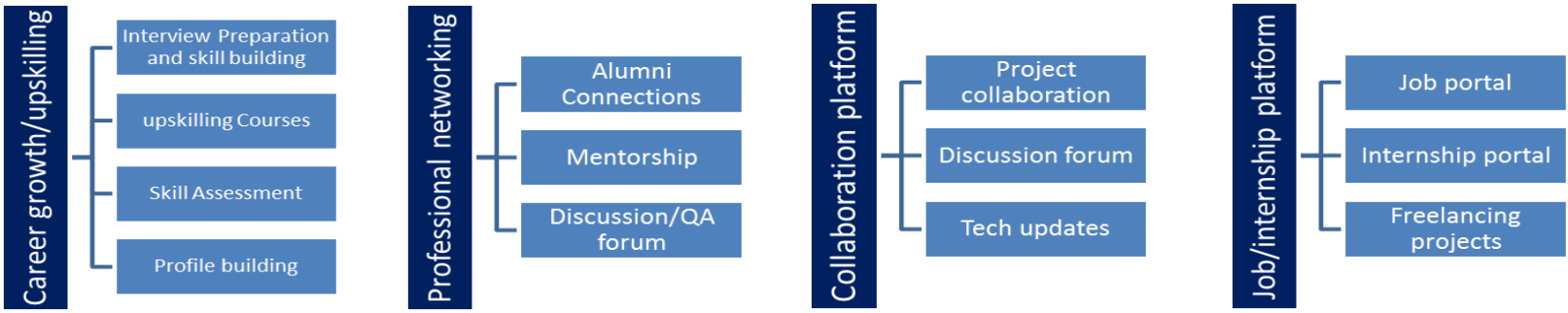
USC is a career development platform that delivers **personalized executive coaching** in a more affordable, scalable and measurable way.



Seeing need of upskilling in self paced manner along-with additional support services e.g. Internship, projects, interaction with Industry experts, Career growth Services

upSkill Campus aiming to upskill 1 million learners in next 5 year

https://www.upskillcampus.com/



2.3 The IoT Academy

The IoT academy is EdTech Division of UCT that is running long executive certification programs in collaboration with EICT Academy, IITK, IITR and IITG in multiple domains.

2.4 Objectives of this Internship program

The objective for this internship program was to

- ☛ get practical experience of working in the industry.
- ☛ to solve real world problems.
- ☛ to have improved job prospects.
- ☛ to have Improved understanding of our field and its applications.
- ☛ to have Personal growth like better communication and problem solving.

2.5 Reference

- Herbert Schildt, **Java: The Complete Reference**, 12th Edition, McGraw-Hill Education.
- Oracle Corporation, **Java Documentation** – https://docs.oracle.com/javase/
- GeeksforGeeks, **Core Java Tutorials** – https://www.geeksforgeeks.org/java/
- W3Schools, **Java Tutorial** – https://www.w3schools.com/java/
- Upskill Campus Internship Learning Resources and Project Guidelines.
- UniConverge Technologies (UCT) Internship Documentation.

2.6 Glossary

Terms	Acronym
Core Java	The fundamental Java programming language used to develop the Expense Tracker application.
Object-Oriented Programming (OOP)	A programming approach based on classes and objects that improves code organization and reusability.
ArrayList	A dynamic collection in Java used to store and manage multiple expense records.
File Handling	The process of reading from and writing data to text files for permanent storage of expenses.
Exception Handling	A Java mechanism that detects and handles runtime errors, ensuring smooth execution of the application.

3 Problem Statement

In the assigned problem statement, the objective was to develop a **Console-Based Expense Tracker** using Core Java that enables users to efficiently manage their daily expenses. The application should allow users to record, update, delete, search, and categorize expenses through a simple menu-driven interface. It should also support expense filtering and generate monthly and category-wise reports for better financial analysis.

The application should ensure persistent storage of expense data using file handling so that users can access their records even after restarting the program. The project aims to provide a simple, reliable, and user-friendly solution for personal expense management while demonstrating Core Java concepts such as Object-Oriented Programming (OOP), Java Collections, File Handling, and Exception Handling.

4 Existing and Proposed solution

- **Existing Solutions**

  Many people manage their daily expenses using notebooks, spreadsheets, or mobile note-taking applications. While these methods are simple, they have several limitations:

  - Manual data entry is time-consuming and prone to errors.
  - It is difficult to organize expenses into categories.
  - Tracking monthly expenses and spending patterns requires manual calculations.
  - Searching, filtering, and updating expense records is inconvenient.
  - Many basic methods do not provide automated reports or secure data storage.

- **Proposed Solution**

  The proposed solution is a **Console-Based Expense Tracker** developed using **Core Java**. The application enables users to record, update, delete, search, and filter expenses through a menu-driven interface. It also allows users to manage expense categories, generate monthly and category-wise reports, and store data permanently using file handling. The system is designed to be simple, reliable, and user-friendly while applying Core Java concepts such as Object-Oriented Programming (OOP), Java Collections, File Handling, and Exception Handling.

- **Value Addition**

  The proposed application offers several improvements over traditional expense management methods:

  - Simple and interactive console-based interface.
  - Organized expense categorization for better tracking.
  - Quick search and filtering of expense records.
  - Automatic monthly, category-wise, and overall expense reports.
  - Permanent data storage using text files.
  - Improved accuracy, reduced manual effort, and easier financial analysis.

4.1 Code submission (Github link): https://github.com/tmhp101106-sudo/upskillcampus/blob/main/ConsoleBasedExpenseTracker.java

4.2 Report submission (Github link) : https://github.com/tmhp101106-sudo/upskillcampus/blob/main/ConsoleBasedExpenseTracker_Haripriya_USC_UCT.pdf

5 Proposed Design/ Model

Oriented Programming (OOP) principles. The application is divided into multiple modules. The proposed solution follows a simple **menu-driven architecture** developed using **Core Java** and Object-responsible for a specific task such as expense management, category management, file handling, report generation, and input validation. This modular design improves code readability, maintainability, and scalability.

- ## Design Flow

Start

- The application starts by displaying the main menu.
- Users select the required operation from the available options.

User Input

- The system accepts user inputs such as expense date, amount, category, and description.
- Input validation is performed to ensure valid and accurate data.

Processing

- Based on the selected option, the application performs operations such as:
    - Add a new expense
    - View all expenses
    - Update existing expenses
    - Delete expenses
    - Manage expense categories
    - Search and filter expenses
    - Generate monthly and category-wise reports

Data Storage

- All expense and category information is stored using text files through Java File Handling.
- The application can load previously saved data whenever it is restarted.

Report Generation

- The system calculates total expenses and generates monthly, category-wise, and overall expense summary reports.
- Users can analyse their spending patterns through these reports.

Final Outcome

- The application provides an organized and efficient way to manage personal expenses.
- Users can easily record, update, search, filter, and analyse their financial data while ensuring data persistence and improved expense management.

Modules Used

- **Main Module:** Displays the menu and controls the application flow.
- **Expense Management Module:** Handles adding, viewing, updating, and deleting expenses.
- **Category Management Module:** Manages expense categories.
- **File Management Module:** Saves and retrieves data from text files.
- **Report Generation Module:** Generates monthly, category-wise, and summary reports.
- **Input Validation Module:** Validates user inputs and handles exceptions.

This modular design ensures that the application is simple, organized, easy to maintain, and can be extended with additional features such as database integration, graphical user interface (GUI), or user authentication in the future.

5.1 High Level Diagram

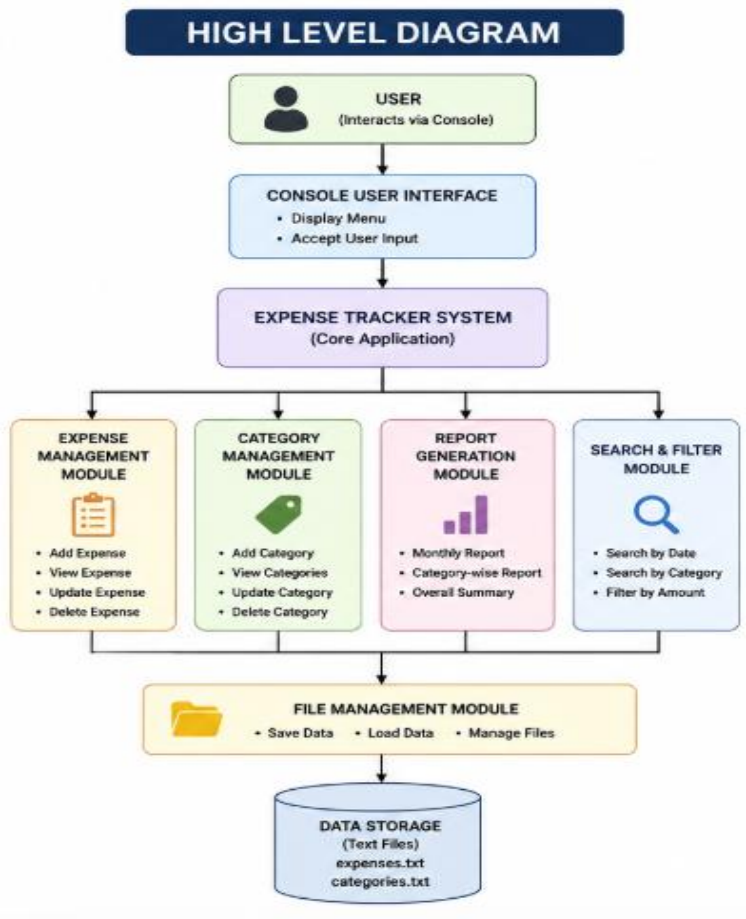


Figure 1: HIGH LEVEL DIAGRAM OF THE SYSTEM

5.2 Low Level Diagram

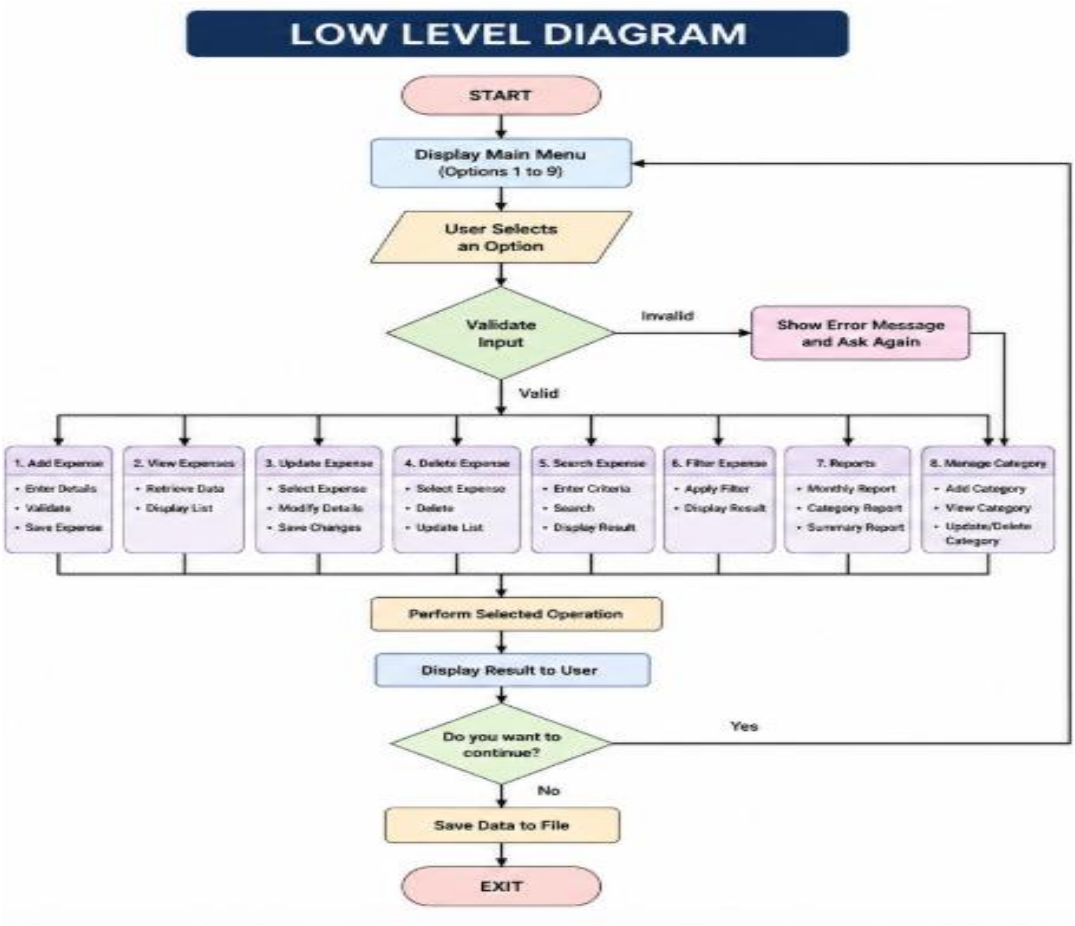


Figure 2: LOW LEVEL DIAGRAM OF THE SYSTEM

5.3 Interfaces

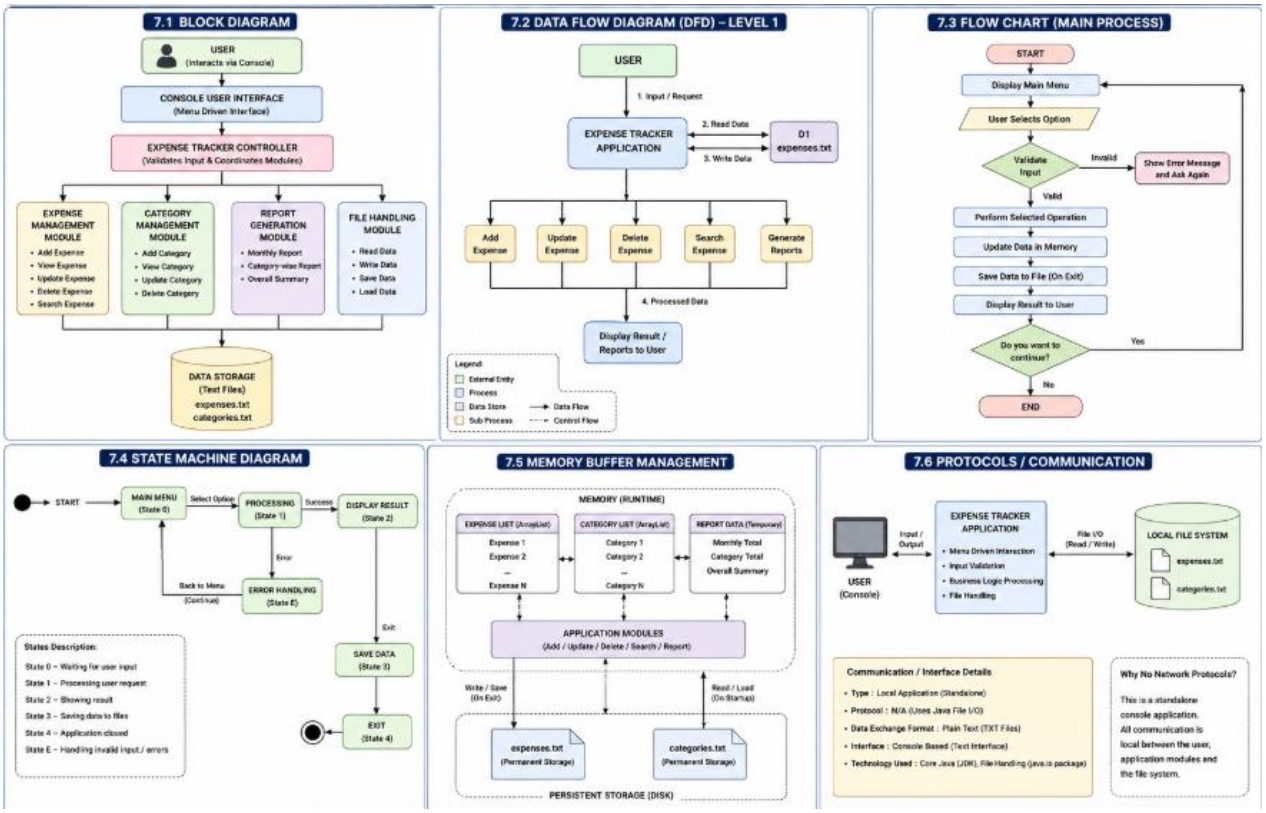


Figure 3: INTERFACES DIAGRAM

6 Performance Test

The performance testing of the **Console-Based Expense Tracker** was carried out to evaluate the application's reliability, efficiency, and usability under normal operating conditions. Since the application is intended for personal expense management, the primary focus was on data accuracy, response time, memory utilization, and data persistence rather than high-end computational performance.

- **Identified Constraints**

- **Memory Usage:** The application stores expense records using an ArrayList, which is suitable for small to medium-sized datasets but may consume more memory as the number of records increases.
- **Processing Speed:** As the application performs linear searches on the expense list, searching and filtering become slower when handling a very large number of records.
- **Data Storage:** Expense records are stored in text files, making them suitable for basic applications but less efficient than databases for handling large datasets.
- **Accuracy:** Incorrect user inputs can affect data quality if not properly validated.
- **Scalability:** The current console-based architecture is designed for a single user and cannot support multiple users simultaneously.

Design Considerations to Handle Constraints

- Used **ArrayList** for efficient management of expense records.
- Implemented **input validation** and **exception handling** to prevent invalid data entry and application crashes.
- Applied **file handling** to ensure permanent storage of expense records after application closure.
- Designed the application using **Object-Oriented Programming (OOP)** principles to improve maintainability and scalability.
- Organized the application into separate modules for expense management, category management, report generation, and file management.

Test Results

- Expense records were added, updated, searched, filtered, and deleted successfully.
- Monthly and category-wise reports were generated accurately.
- Data was successfully saved and retrieved using text files.

- The application responded quickly for normal datasets (approximately 100–500 expense records).
- Input validation successfully prevented invalid entries and improved application stability.

- **Recommendations**
  - Replace text file storage with a relational database such as MySQL for better scalability.
  - Use optimized searching and sorting algorithms for improved performance with large datasets.
  - Develop a graphical user interface (GUI) for better usability.
  - Add multi-user support with authentication and cloud-based storage for enterprise-level usage.

6.1 Test Plan/ Test Cases

Test Case	Test Description	Expected Result	Status
TC01	Add a new expense	Expense added successfully	Passed
TC02	View all expenses	Displays all saved expenses	Passed
TC03	Update an expense	Expense details updated correctly	Passed
TC04	Delete an expense	Selected expense removed	Passed
TC05	Search by category	Displays matching expenses	Passed
TC06	Search by date	Displays expenses for selected date	Passed
TC07	Filter by amount	Displays matching expense records	Passed
TC08	Generate monthly report	Monthly total calculated correctly	Passed
TC09	Generate category-wise report	Correct category totals displayed	Passed
TC10	Save and reload data	Data successfully retained after restart	Passed
TC11	Invalid input handling	Displays appropriate error message	Passed

6.2 Test Procedure

- Launched the Expense Tracker application.
- Added multiple expense records with different dates, categories, and amounts.
- Verified that the expenses were displayed correctly.
- Updated and deleted selected expense records.
- Tested search and filtering functionalities using different criteria.
- Generated monthly, category-wise, and overall expense reports.
- Saved the expense records to text files and restarted the application.
- Loaded the saved data and verified that all records were restored correctly.
- Entered invalid inputs to test input validation and exception handling.
- Verified that all modules functioned correctly without application crashes.

6.3 Performance Outcome

The Console-Based Expense Tracker successfully met all functional requirements. The application accurately managed expense records, generated reports, and stored data persistently using file handling. Performance remained stable and responsive for small and medium-sized datasets. The implemented input validation and exception handling improved application reliability and prevented invalid data entry. Although the application is suitable for personal expense management, future enhancements such as database integration, optimized searching, and a graphical user interface can further improve its scalability, performance, and usability for larger real-world applications.

7 My learnings

The four-week internship provided me with valuable practical exposure to **Core Java** by developing a **Console-Based Expense Tracker** application. Throughout the project, I applied Object-Oriented Programming (OOP), Java Collections (ArrayList), File Handling, and Exception Handling to build a functional application. This hands-on experience strengthened my programming fundamentals and helped me understand the complete software development lifecycle, from planning and design to implementation, testing, and documentation.

During the internship, I enhanced my technical, analytical, and problem-solving skills by implementing features such as expense management, category management, report generation, data persistence, and input validation. I also learned the importance of writing modular, maintainable code, debugging errors, and testing applications to ensure reliability. Working on a real-world project improved my confidence in developing Java applications independently and deepened my understanding of software design principles.

This internship has significantly contributed to my personal and professional growth by improving my coding skills, logical thinking, time management, and project planning abilities. The practical knowledge gained through this project has prepared me for future academic projects, advanced Java development, and software engineering roles. Overall, the internship has built a strong foundation for my career in the IT industry and motivated me to continue learning and enhancing my technical expertise.

8 Future work scope

Although the Console-Based Expense Tracker successfully meets its objectives, several enhancements can be implemented in the future to make the application more advanced and user-friendly.

- Develop a **Graphical User Interface (GUI)** using Java Swing or JavaFX for a better user experience.
- Integrate a **database** such as MySQL instead of text files for improved data storage and management.
- Add **user authentication** with login and registration to support multiple users securely.
- Implement **budget planning** and notifications to alert users when spending exceeds a predefined limit.
- Generate **graphical reports and charts** to visualize monthly and category-wise expenses.
- Add features such as **expense export** to PDF or Excel and data backup options.
- Include **search by keywords**, advanced filters, and sorting for quicker expense analysis.
- Develop a **mobile or web-based version** of the application for easy access across different devices.

These enhancements would improve the functionality, scalability, and usability of the application, making it more suitable for real-world personal finance management.

Thank You